CONCURRENT DATA PROCESSING ENGINE

Product specification, exact problem statement, API examples, architecture, and implementation plan

Project type: C++ systems programming / data processing
Target: A small, local, multi-threaded batch engine for CSV data
Design goal: Correct, measurable parallelism before advanced features

This document defines what to build, what the finished product should do, how a user will invoke it, and how to verify that it works. It is a specification for implementation—not a claim that the engine has already been built.

Version 1.0 • 26 October 2026

1. Executive summary

Build a command-line data processing engine in modern C++ that reads a CSV file, splits its records into batches, schedules work across a fixed pool of worker threads, applies supported operations, and writes deterministic output. The engine should provide a simple user-facing interface while exposing enough internal structure to demonstrate sound systems design.

The product is inspired by the local execution model of larger data-processing systems: partition work, process partitions independently, and combine partial results. It is not a distributed system and does not attempt to reproduce all of Apache Spark.

Exact problem statement

Implement a reusable, local, multi-threaded batch processing engine in C++20. Given a CSV input file, a schema or column selection, an operation, and a worker count, the program must parse records, partition them into bounded chunks, schedule chunk-level tasks on a fixed-size thread pool, process tasks concurrently, combine results where required, and emit a result file or summary. It must preserve correctness relative to a sequential reference implementation, report processing statistics, handle malformed input and task failures predictably, and include automated tests and reproducible benchmarks.

Why this is a worthwhile project

- Concurrency: threads, mutexes, condition variables, task queues, shutdown, and exception propagation.
- C++ design: ownership, RAII, value types, interfaces, templates or callable tasks, and clear header/source boundaries.
- Data processing: chunking, filtering, mapping, grouping, aggregation, and output serialization.
- Performance engineering: sequential baseline, throughput, speedup, scaling, and bottleneck analysis.
- Software quality: deterministic tests, error handling, build configuration, documentation, and reproducible measurements.

Primary user story

As a user, I can point the engine at a CSV file, choose an operation and number of worker threads, and receive a correct output plus a concise report of how much data was processed and how long it took—without needing to manage threads myself.

Success means correctness first, then measurable performance. A multi-threaded implementation that is faster on one test but occasionally loses records or produces inconsistent totals is not acceptable.

2. Scope and product requirements

Version 1 must be deliberately bounded. Finish a small, reliable engine before adding query languages, distributed execution, GPU processing, or a complicated optimizer.

Required features

ID	Requirement	Acceptance condition
FR-01	Read CSV input	Read header and rows; detect unreadable files and malformed records.
FR-02	Chunk records	Create bounded batches; final batch may be smaller than the configured chunk size.
FR-03	Fixed thread pool	Start N worker threads once and reuse them across tasks.
FR-04	Task queue	Workers safely wait for work and wake when tasks arrive.
FR-05	Operations	Support at least filter, map/transform, and aggregate/reduce.
FR-06	Output	Write results or aggregate summaries to a specified path.
FR-07	Statistics	Report input records, output records where applicable, elapsed time, throughput, and worker count.
FR-08	Failure handling	Surface file, parse, task, and output errors; never silently drop failed tasks.
FR-09	Repeatable results	For deterministic operations, output and aggregates match the sequential reference.
FR-10	Tests and benchmarks	Provide correctness tests and a sequential-vs-parallel benchmark harness.

Non-functional requirements

- **Language/build:** C++20, CMake, and a compiler with standard library thread support.
- **Portability:** Avoid platform-specific threading APIs in the core engine; use the C++ standard library.
- **Bounded memory:** Process data in chunks rather than requiring the whole file to be stored in memory. Version 1 may initially load a chunk before scheduling it.
- **Correctness:** No data races; no lost or duplicated tasks; clean shutdown; deterministic aggregate results within the stated numeric policy.
- **Observability:** Print clear errors and performance counters; do not present illustrative numbers as measured benchmarks.
- **Maintainability:** Separate CSV parsing, scheduling, processing, aggregation, and CLI concerns.

Explicitly out of scope for version 1

- Distributed execution across machines, fault-tolerant cluster scheduling, and network transport.
- SQL parsing, a query optimizer, joins, indexes, or a database storage engine.
- GPU/CUDA acceleration, SIMD-specific optimization, and a custom memory allocator.
- Real-time streaming, exactly-once event processing, and persistent job recovery.
- A general-purpose dataframe API with full type inference and complex nested types.

3. User experience and example usage

The finished product should be usable from a terminal. A user specifies the input, output, operation, relevant columns, and worker count. The CLI validates arguments, executes the pipeline, and prints a summary.

Example A — filter records

```
./build/engine filter \
  --input data/transactions.csv \
  --output out/large_transactions.csv \
  --where transaction_amount gt 10000 \
  --threads 4 \
  --chunk-size 50000
```

Meaning: read the CSV, retain rows whose transaction_amount is greater than 10,000, process chunks using four workers, and write the retained rows to the output CSV.

Example B — aggregate a numeric column

```
./build/engine aggregate \
  --input data/transactions.csv \
  --column transaction_amount \
  --operation sum \
  --threads 4
```

Example output (format illustration only)

```
Concurrent Data Processing Engine
Input:             data/transactions.csv
Operation:         aggregate(sum)
Column:            transaction_amount
Input records:     10,000,000
Worker threads:    4
Chunk size:        50,000
Elapsed time:      1.42 s
Throughput:        7,042,253 records/s
Result:            8,193,420,115.50
Status:            success
```

The values above are placeholders to show the intended interface. The implemented engine must print measured values from the actual run.

Example C — use the C++ API

```
#include "engine/engine.hpp"
#include <iostream>

int main() {
    engine::EngineConfig config;
    config.worker_count = 4;
    config.chunk_size = 50'000;

    engine::Engine engine(config);

    auto result = engine.aggregate_csv(
        "data/transactions.csv",
        "transaction_amount",
        engine::Aggregate::Sum
    );

    std::cout << "Rows: " << result.rows_processed << '\\n';
    std::cout << "Sum: " << result.numeric_result << '\\n';
    std::cout << "Elapsed: " << result.elapsed_seconds << " s\\n";
}
```

This is an illustrative target API, not a pre-existing library interface. Keep the public API small and stable; implementation details such as mutexes and queues should remain private.

4. In-depth product behavior

4.1 Input and CSV semantics

- The first row is treated as a header unless the user explicitly chooses a headerless mode.
- Columns are addressed by header name for CLI operations. Unknown column names produce a clear error.
- The parser must handle quoted fields and escaped quotes; do not implement CSV by blindly splitting each line on commas.
- Version 1 should define a simple type policy: preserve raw fields as strings for filter/output, and parse selected numeric columns strictly for arithmetic.
- A malformed row should either fail the job with its row number or be skipped only under an explicit option. Never silently skip malformed input.

4.2 Chunking and task boundaries

A chunk is a batch of records processed as one task. Chunk size is a tuning parameter: tiny chunks increase scheduling overhead; huge chunks increase memory use and can create load imbalance. Start with a fixed record count per chunk and expose the setting in the CLI.

```
CSV reader
  -> chunk 0 -> task queue -> worker -> partial result 0
  -> chunk 1 -> task queue -> worker -> partial result 1
  -> chunk 2 -> task queue -> worker -> partial result 2
  -> ...                          -> aggregator -> final result
```

4.3 Thread-pool contract

- The pool owns a fixed number of worker threads for its lifetime.
- A worker waits on a condition variable while the queue is empty and the pool is not stopping.
- Submitting a task pushes it under the queue mutex and notifies a waiting worker.
- Shutdown sets a stopping flag, wakes all workers, and joins every thread.
- The pool must define whether shutdown drains queued work or cancels pending work. Version 1 should drain already accepted tasks, then exit.
- Task exceptions are captured and delivered to the caller/future; they must not escape the worker entry point and terminate the process.

4.4 Operation semantics

Operation	Definition	Parallel strategy
Filter	Keep rows satisfying a predicate.	Each chunk filters independently; output chunks are written in original chunk order if stable order is promised.
Map/transform	Convert selected fields or derive a field.	Each row/chunk transforms independently; output order is preserved by chunk index.
Reduce/aggregate	Combine values into a scalar or grouped summary.	Each chunk computes a partial result; a final reduction combines partials.
Group-by (stretch)	Group by key and compute aggregates.	Workers produce local maps; merge local maps after task completion.

4.5 Determinism and numeric behavior

Parallel tasks can complete in a different order from submission. Therefore, never use completion order as the output order when stable row ordering is part of the contract. Floating-point addition is not associative, so parallel sums can differ slightly from sequential sums due to rounding. Define a tolerance for floating-point tests or use integer/fixed-point data when exact financial arithmetic is required.

5. Architecture and responsibilities

Keep components independently testable. The engine orchestrates them; the thread pool should not know what a CSV row or aggregate means.

Component	Owns / does	Should not do
CLI	Parse arguments, validate options, choose operation, print summary.	Implement thread synchronization or CSV parsing.
CsvReader / CsvWriter	Parse and serialize records; report row/column errors.	Schedule tasks or maintain worker threads.
DataChunk	Own a batch of parsed records and a stable chunk ID.	Share mutable row storage unsafely between workers.
ThreadPool	Own threads, task queue, condition variable, shutdown state.	Know domain-specific operations.
Processor	Apply filter/map logic to one chunk.	Mutate global output structures without synchronization.
Aggregator	Merge partial results after tasks complete.	Depend on task completion order for correctness.
Metrics	Measure elapsed wall time and count records/bytes.	Claim speedups without a valid baseline.

Suggested source tree

```
concurrent-engine/
■■■ CMakeLists.txt
■■■ README.md
■■■ include/engine/
■   ■■■ engine.hpp
■   ■■■ engine_config.hpp
■   ■■■ data_chunk.hpp
■   ■■■ csv_reader.hpp
■   ■■■ csv_writer.hpp
■   ■■■ thread_pool.hpp
■   ■■■ processor.hpp
■   ■■■ aggregator.hpp
■   ■■■ metrics.hpp
■■■ src/
■   ■■■ main.cpp
■   ■■■ csv_reader.cpp
■   ■■■ csv_writer.cpp
■   ■■■ thread_pool.cpp
■   ■■■ processor.cpp
■   ■■■ engine.cpp
■■■ tests/
■   ■■■ test_csv.cpp
■   ■■■ test_thread_pool.cpp
■   ■■■ test_operations.cpp
■   ■■■ test_engine.cpp
■■■ benchmarks/
■   ■■■ benchmark_engine.cpp
■■■ data/
    ■■■ sample_transactions.csv
```

Key data-flow decision

Use one task per chunk. Each task owns its input chunk and returns a result object or writes to a unique indexed result slot. Avoid having all workers append to one shared vector protected by a single mutex: that design is easy to write but can serialize the hot path and obscure ordering guarantees.

Ownership rule

Prefer RAII and standard containers. The thread pool owns its threads; the queue owns queued task wrappers; a task owns or safely shares its chunk; partial results are owned by futures or stable result slots. Avoid raw owning pointers unless a specific low-level need is justified.

6. Concurrency: what must be correct

A thread pool is more than a vector of std::thread objects. The difficult part is coordinating shared state and shutdown without missed wake-ups, data races, deadlocks, or lost work.

Core shared state

```
std::mutex queue_mutex;
std::condition_variable work_available;
std::queue<Task> tasks;
bool stopping = false;
```

Worker loop — conceptual pseudocode

```
while (true) {
    Task task;

    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        work_available.wait(lock, [&] {
            return stopping || !tasks.empty();
        });

        if (stopping && tasks.empty())
            return;                 // drain queue, then exit

        task = std::move(tasks.front());
        tasks.pop();
    }                               // unlock before doing work

    try {
        task();                     // do not hold queue lock here
    } catch (...) {
        // Capture/report exception through task result or future.
    }
}
```

Why each line matters

- **Predicate wait:** condition variables can wake spuriously. Always re-check a predicate while holding the mutex.
- **Short critical section:** remove the task under the lock, then unlock before executing it. Otherwise one slow task blocks every worker from taking work.
- **Stopping plus empty queue:** workers exit only when shutdown has been requested and no accepted tasks remain.
- **Join:** destruction must wait for worker threads to finish before shared state is destroyed.
- **Exception channel:** catch exceptions at the worker boundary and store them in a future/promise or task result.

Important edge cases to test

- Zero workers requested; define this as an error or normalize it to one worker.
- Empty input, header-only CSV, and a final partial chunk.
- More workers than chunks and more tasks than workers.
- Task throws an exception; subsequent tasks still run and the caller sees the failure.
- Pool is destroyed with queued work remaining.
- Two independent engine instances run at the same time.
- CSV output path cannot be opened or disk write fails.

Do not begin by optimizing lock contention. First implement a simple correct queue, prove its behavior with tests, then profile. A work-stealing scheduler or lock-free queue is not necessary for this project.

7. API design and implementation guidance

A small configuration object makes defaults explicit and prevents a long constructor argument list.

```
struct EngineConfig {
    std::size_t worker_count = 4;
    std::size_t chunk_size = 50'000;
};

struct RunStats {
    std::uint64_t rows_read = 0;
    std::uint64_t rows_written = 0;
    double elapsed_seconds = 0.0;
    double records_per_second = 0.0;
};

enum class Aggregate {
    Count,
    Sum,
    Mean,
    Min,
    Max
};
```

Possible public interface

```
class Engine {
public:
    explicit Engine(EngineConfig config);

    RunStats filter_csv(
        const std::filesystem::path& input,
        const std::filesystem::path& output,
        const FilterSpec& filter);

    AggregateResult aggregate_csv(
        const std::filesystem::path& input,
        std::string_view column,
        Aggregate operation);
};
```

Treat this as a design target, not a requirement to use exactly these signatures. Choose interfaces after you understand the data model and error policy.

Recommended task abstraction

Use a task wrapper that can carry arbitrary callable work and return a result to the submitting thread. A standard approach is to wrap a callable in std::packaged_task, place a void()-compatible wrapper in the queue, and return its std::future. This avoids manually implementing promise state for every task.

```
template<class F, class... Args>
auto submit(F&& f, Args&&... args)
    -> std::future<std::invoke_result_t<F, Args...>>;

// Implementation sketch:
// 1. Create shared packaged_task for the callable.
// 2. Obtain its future.
// 3. Push a wrapper that invokes the packaged_task.
// 4. Notify one worker.
// 5. Return the future to the caller.
```

Error model

- Use exceptions for unrecoverable setup or I/O failures, or define a project-wide expected/result type—do not mix policies arbitrarily.
- Include context in error messages: file path, row number, column name, or task identifier.
- Do not catch and discard exceptions. If a task fails, the overall operation should fail visibly unless an explicit continue-on-error policy is selected.

8. Testing and performance evaluation

A concurrency project needs evidence. The benchmark is not just a speed number: it should help explain whether parallelism is useful for the chosen workload.

Correctness tests

Test group	Examples	Expected result
CSV parsing	Quoted comma, escaped quote, empty field, malformed row.	Correct parse or explicit, contextual error.
Operations	Filter boundary, map conversion, sum/count/mean.	Matches hand-calculated expected output.
Thread pool	Submit many tasks, throw in one task, shut down.	No lost tasks; errors observed; clean exit.
Parallel equivalence	Run same dataset sequentially and with 1/2/4 workers.	Same rows and exact/tolerance-compliant aggregates.
Boundary sizes	0, 1, chunk_size-1, chunk_size, chunk_size+1 rows.	Correct result with no missing/duplicate records.
Output	Unwritable destination, empty result, existing output.	Clear failure or documented overwrite behavior.

Benchmark protocol

- Create or obtain reproducible datasets at several sizes, for example 100k, 1M, and 10M rows.
- Implement a simple sequential baseline that performs the same parsing and operation semantics.
- Warm up where appropriate and run each configuration multiple times; report median and spread, not only the best run.
- Compare worker counts such as 1, 2, 4, and the number of available logical CPUs. Do not assume more threads are always faster.
- Keep input, output, compiler build type, chunk size, and operation constant when comparing worker counts.
- Separate end-to-end timing from processing-only timing if CSV I/O dominates.
- Record machine/CPU, compiler, build flags, dataset size, and command used.

Metrics to calculate

Metric	Definition	Interpretation
Throughput	records processed / elapsed seconds	How much work completes per second.
Speedup	T_sequential / T_parallel	How much faster parallel execution is.
Parallel efficiency	speedup / worker_count	How effectively added workers are used.
Scaling curve	speedup vs. worker count	Shows diminishing returns or regressions.

Example: if sequential execution takes 8 seconds and four-worker execution takes 3 seconds, speedup = 8 / 3 = 2.67× and parallel efficiency = 2.67 / 4 = 66.7%. Use measured values only.

Performance pitfalls to discuss in README

- I/O-bound workloads may not improve much with more worker threads.
- Parsing, allocation, cache misses, and output serialization may dominate runtime.
- Tiny chunks create queue and scheduling overhead; large chunks may reduce load balance.
- A shared aggregation lock can serialize work; prefer per-chunk partial results and a final merge.
- On a laptop, thermal limits and background activity can make measurements noisy.

9. Implementation plan

Milestone	Build	Done when
M1 — sequential baseline	CSV reader/writer, numeric parsing, one operation, basic CLI.	Correct output on small fixtures; clear parse errors.
M2 — chunk model	DataChunk, chunk IDs, fixed-size chunking.	Boundary-size tests pass.
M3 — thread pool	Workers, queue, mutex, condition variable, shutdown, futures.	Stress tests pass; no hangs or lost tasks.
M4 — parallel filter/map	One task per chunk, indexed result slots, ordered output.	Parallel output matches sequential output.
M5 — parallel aggregation	Per-chunk partials and final reduction.	Aggregates match reference within defined numeric tolerance.
M6 — metrics and benchmarks	Timing, throughput, speedup, repeated runs.	Benchmark script and documented environment.
M7 — polish	CLI validation, error paths, README, examples, CMake tests.	A new user can build and run it from the README.

Definition of done

- A clean build works from documented instructions using CMake.
- All automated tests pass on the target machine.
- Sequential and parallel versions agree on the same deterministic test fixtures.
- The engine handles empty files, partial chunks, invalid columns, malformed rows, and task exceptions predictably.
- A benchmark can be reproduced with a documented command and reports measured numbers.
- The README explains architecture, thread safety, numeric limitations, design decisions, and known limitations.
- No performance claim is made without showing the workload and measurement method.

Suggested initial constraints

Use C++20, a fixed thread pool, a bounded chunk size, CSV input, and three operations (filter, map/transform, aggregate). Start with numeric aggregates and stable row-preserving filter output. Add group-by only after the base engine is correct.

10. Evaluation rubric and final checklist

Area	Weight	What strong work looks like
Correctness	30%	No missing/duplicated work; deterministic output; strong edge-case tests.
Concurrency design	25%	Clear queue/worker lifecycle, correct synchronization, exception propagation, clean shutdown.
Architecture and C++	15%	Separation of concerns, RAII, good interfaces, ownership clarity, modern C++.
Performance analysis	15%	Fair sequential baseline, repeated measurements, speedup/efficiency, bottleneck discussion.
Usability and docs	10%	Clear CLI, useful errors, build instructions, example data and commands.
Engineering polish	5%	Readable code, formatting, CI or repeatable tests, sensible scope.

Before calling the project complete

- Can you explain why the condition-variable wait uses a predicate?
- Can you show that the queue mutex is not held while a task runs?
- Can you explain how the pool drains tasks and shuts down?
- Can you prove that each chunk is processed exactly once in normal execution?
- Can you explain how task exceptions reach the caller?
- Can you show output equality against the sequential implementation?
- Can you explain why speedup is less than worker count and identify the likely bottleneck?
- Can another developer build and run the project without asking you for missing steps?

Recommended repository README outline

```
# Concurrent Data Processing Engine
## Overview and goals
## Features and non-goals
## Requirements and build instructions
## CLI usage and examples
## Architecture and data flow
## Thread-pool design and shutdown semantics
## Correctness guarantees and numeric policy
## Tests
## Benchmark methodology and results
## Limitations and future work
```

Final product definition

The desired product is a small but credible systems project: a user-facing CSV processing tool backed by a reusable C++ thread pool, chunk-based parallel operations, safe aggregation, explicit error behavior, tests, and defensible performance measurements. Its strongest feature is not a large number of options; it is that the concurrency is correct, the API is understandable, and the performance claims are supported by evidence.

End of specification